SRI CHANDRASEKHARENDRA SARASWATHI VISWA

MAHAVIDYALAYA

(UNIVERSITYESTABLISHEDUNDERSECTION3 OFUGCACT1956)
ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Name : M.Vaidhya Nadeswar Reddy

Reg. No : 11249A239

Class : **II B.E. (CSE)**

Course Code: BCSF244AI2

Course Name: Data science for tool kit lab

SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that this is the bonafide record of work done by Mr/Ms. M.Vaidhya Nadeswar Reddy ;

with Reg. No: 11249A239 of II-B.E.(CSE) during the academic year 2025 – 2026.

Station: Enathur
Date: 28.04.2026

Staff-in-charge Head of the Department

Submitted for the Practical examination held on 28.04.2026 ;

Examiner-1 Examiner-2

EXP.NO.1.1 NUMPY OPERATIONS

AIM: To perform basic array operations using NumPy library.

PROGRAM:

```
import numpy as np
print(
    "1D Array:\n",np.array([1,2,3,4,5]),"\n\n"
    "2D Array:\n",np.array([[1,2,3],[4,5,6]]),"\n\n"
    "Zeros:\n",np.zeros((3,3)),"\n\n"
    "Ones:\n",np.ones((2,4)),"\n\n"
    "Identity:\n",np.eye(3),"\n\n"
    "Rnage:\n",np.arange(0,10,2)
)
```

OUTPUT:

```
1D Array:
 [1 2 3 4 5]

2D Array:
 [[1 2 3]
 [4 5 6]]

Zeros:
 [[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]

Ones:
 [[1. 1. 1. 1.]
 [1. 1. 1. 1.]]

Identity:
 [[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]

Rnage:
 [0 2 4 6 8]
```

Result: Successfully performed basic array operations using NumPy.

EXP.NO.1.2 PANDAS PROGRAM

AIM: To perform data manipulation and analysis using Pandas DataFrames.

PROGRAM:

```
import pandas as pd
data ={
    'Name':['Alic', 'Bob', 'Charlie'],
    'Age':[25,30, 35],
    'City':['New York', 'Paris', 'London']
}
df = pd.DataFrame(data)
print(df.head())
print(df.info())
print(df.describe())
ages = df['Age']
above_25= df[df['Age']>25]
row_0= df.iloc[0]
specific_val = df.loc[0, 'City']
print(df.isnull().sum())
df_clean = df.dropna()
df_filled = df.fillna(0)
```

OUTPUT:

```
      Name  Age      City
0     Alic   25  New York
1      Bob   30     Paris
2  Charlie   35    London
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Name    3 non-null      object
 1   Age     3 non-null      int64
 2   City    3 non-null      object
dtypes: int64(1), object(2)
memory usage: 204.0+ bytes
None
        Age
count   3.0
mean   30.0
std     5.0
min    25.0
25%    27.5
50%    30.0
75%    32.5
max    35.0
Name    0
Age     0
City    0
dtype: int64
```

Result: Data manipulation and analysis using Pandas completed successfully.

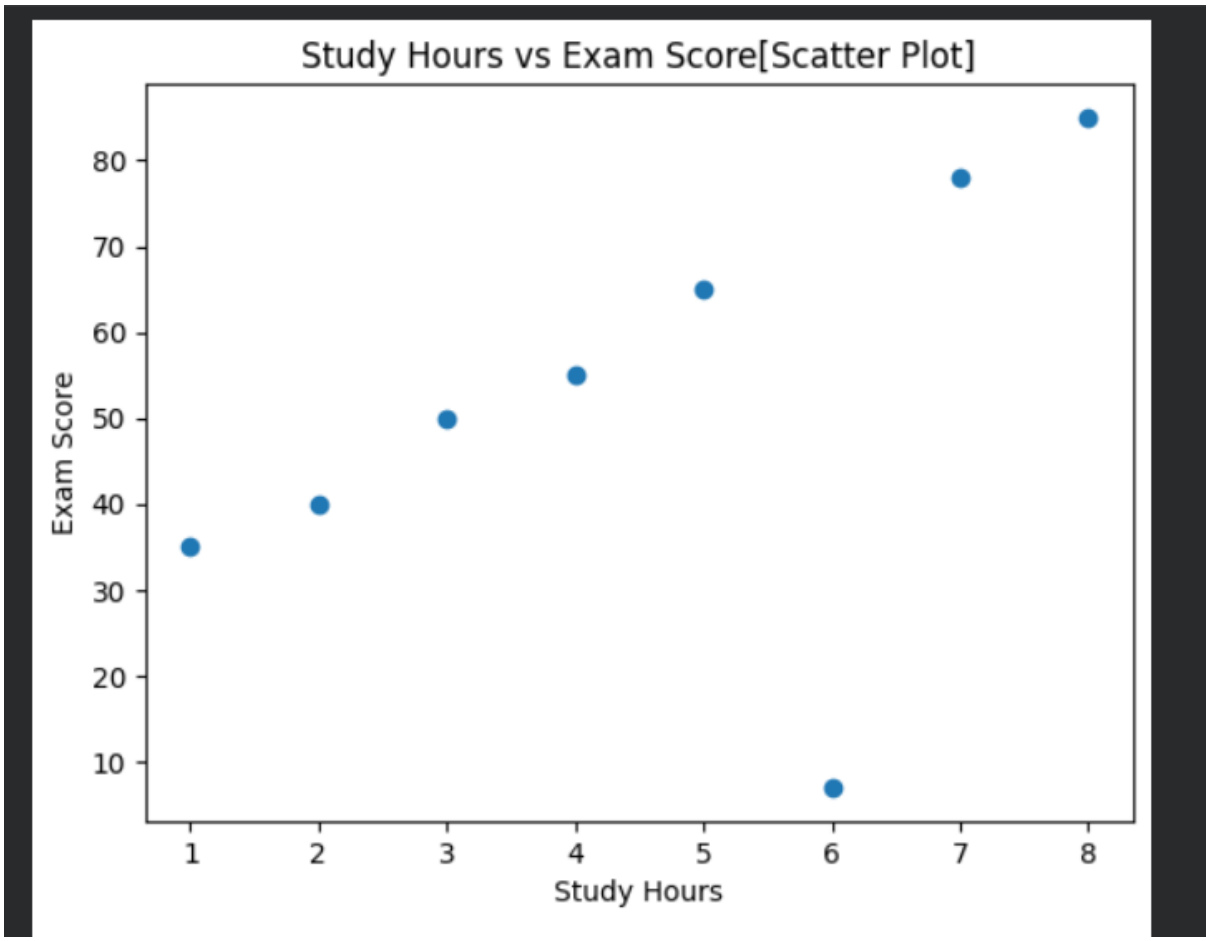
EXP.NO.1.3 Matplotlib Plotting

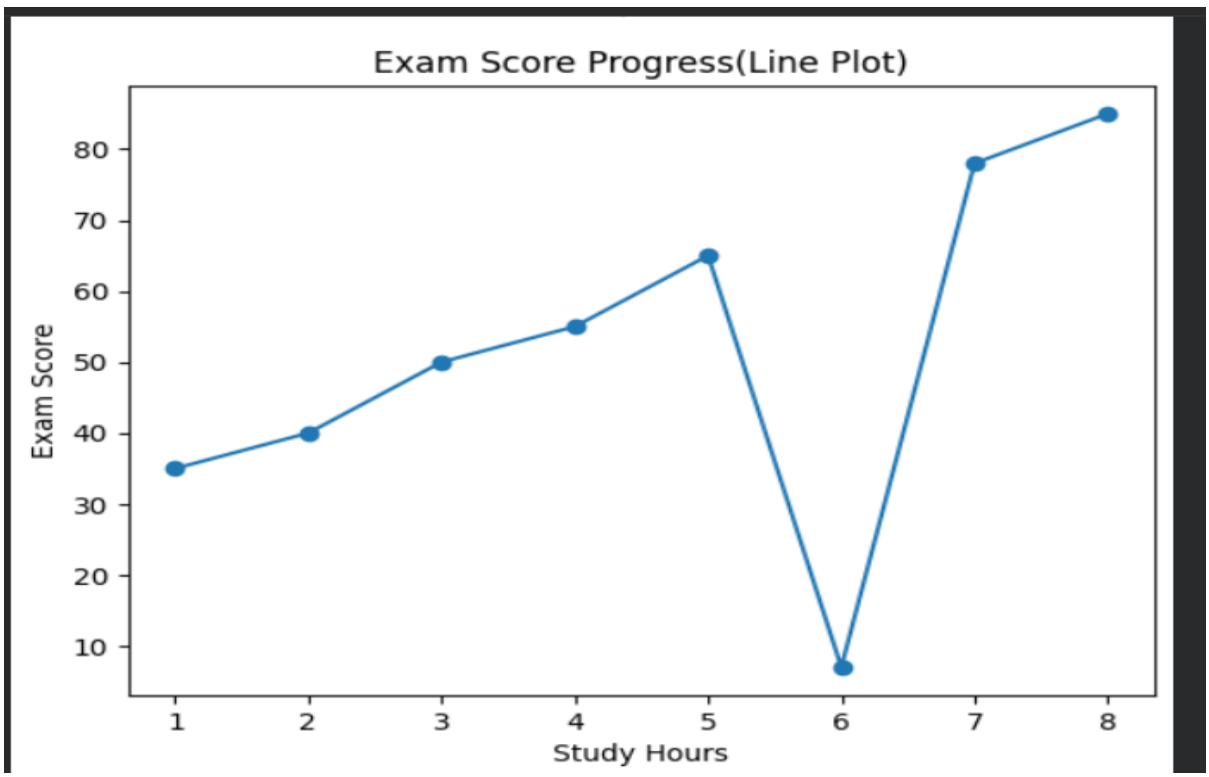
Aim: To visualize data using different plots with Matplotlib.

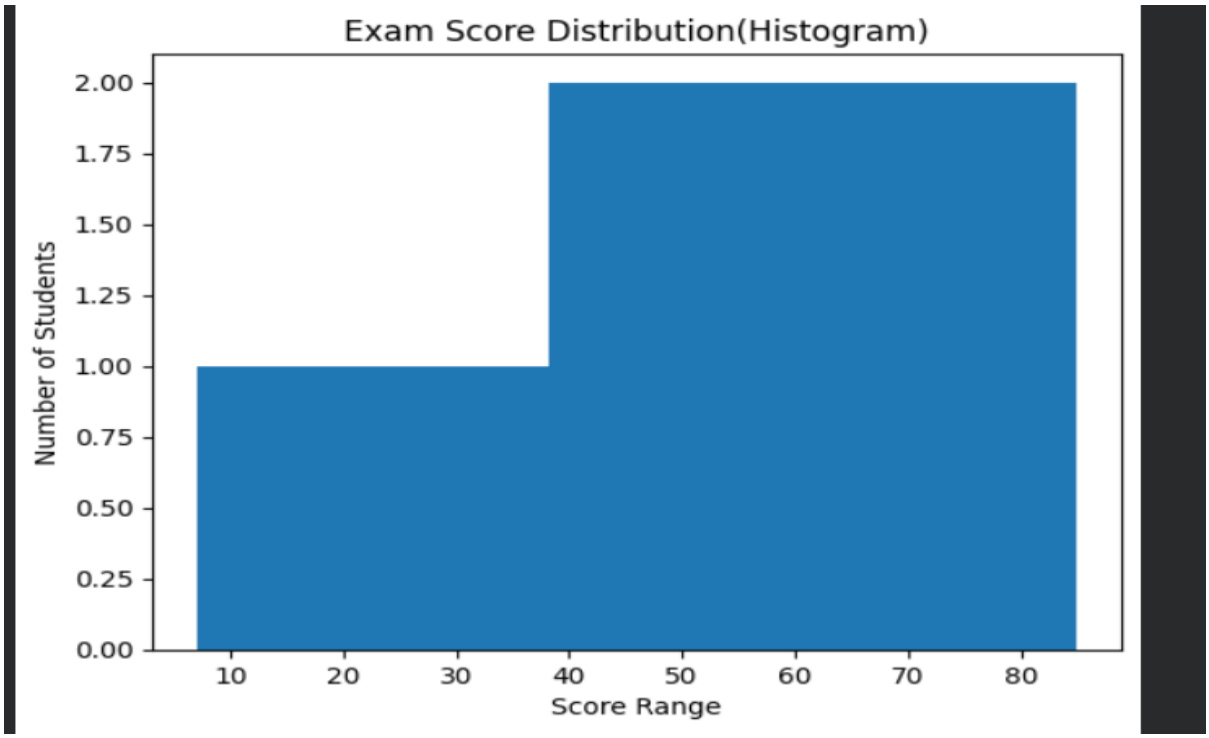
PROGRAM:

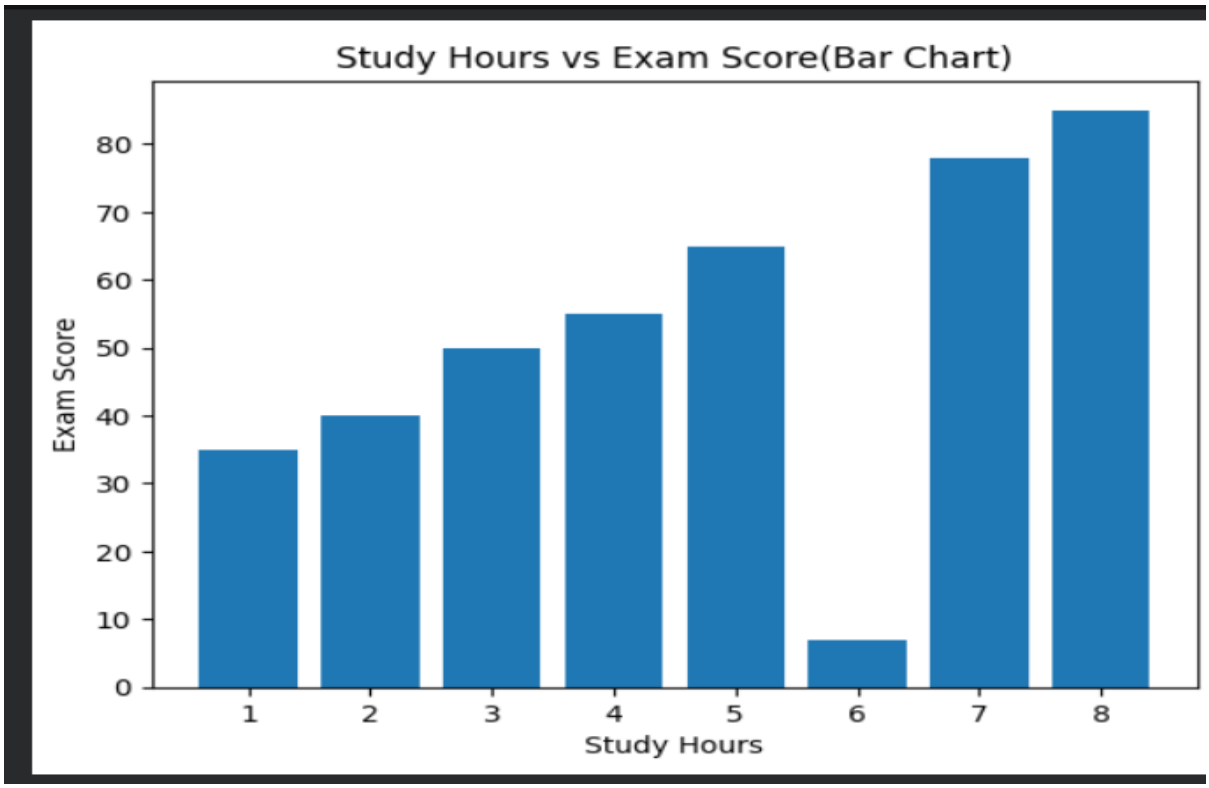
```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
data={
    'StudyHours':[1,2,3,4,5,6,7,8],
    'ExamScore':[35,40,50,55,65,7,78,85]
}
df=pd.DataFrame(data)
plt.scatter(df['StudyHours'],df['ExamScore'])
plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.title("Study Hours vs Exam Score[Scatter Plot]")
plt.show()
plt.plot(df['StudyHours'],df['ExamScore'],marker='o')
plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.title("Exam Score Progress(Line Plot)")
plt.show()
plt.hist(df['ExamScore'],bins=5)
plt.xlabel("Score Range")
plt.ylabel("Number of Students")
plt.title("Exam Score Distribution(Histogram)")
plt.show()
plt.bar(df['StudyHours'],df['ExamScore'])
plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.title("Study Hours vs Exam Score(Bar Chart)")
plt.show()
```

OUTPUT:


Study Hours vs Exam Score[Scatter Plot]
Exam Score
Study Hours
80
70
60
50
40
30
20
10
1
2
3
4
5
6
7
8


Exam Score Progress(Line Plot)
Exam Score
Study Hours
80
70
60
50
40
30
20
10
1
2
3
4
5
6
7
8





Result: Graphs and plots were generated successfully using Matplotlib.

EXP.NO.1.4. Scikit-learn Program

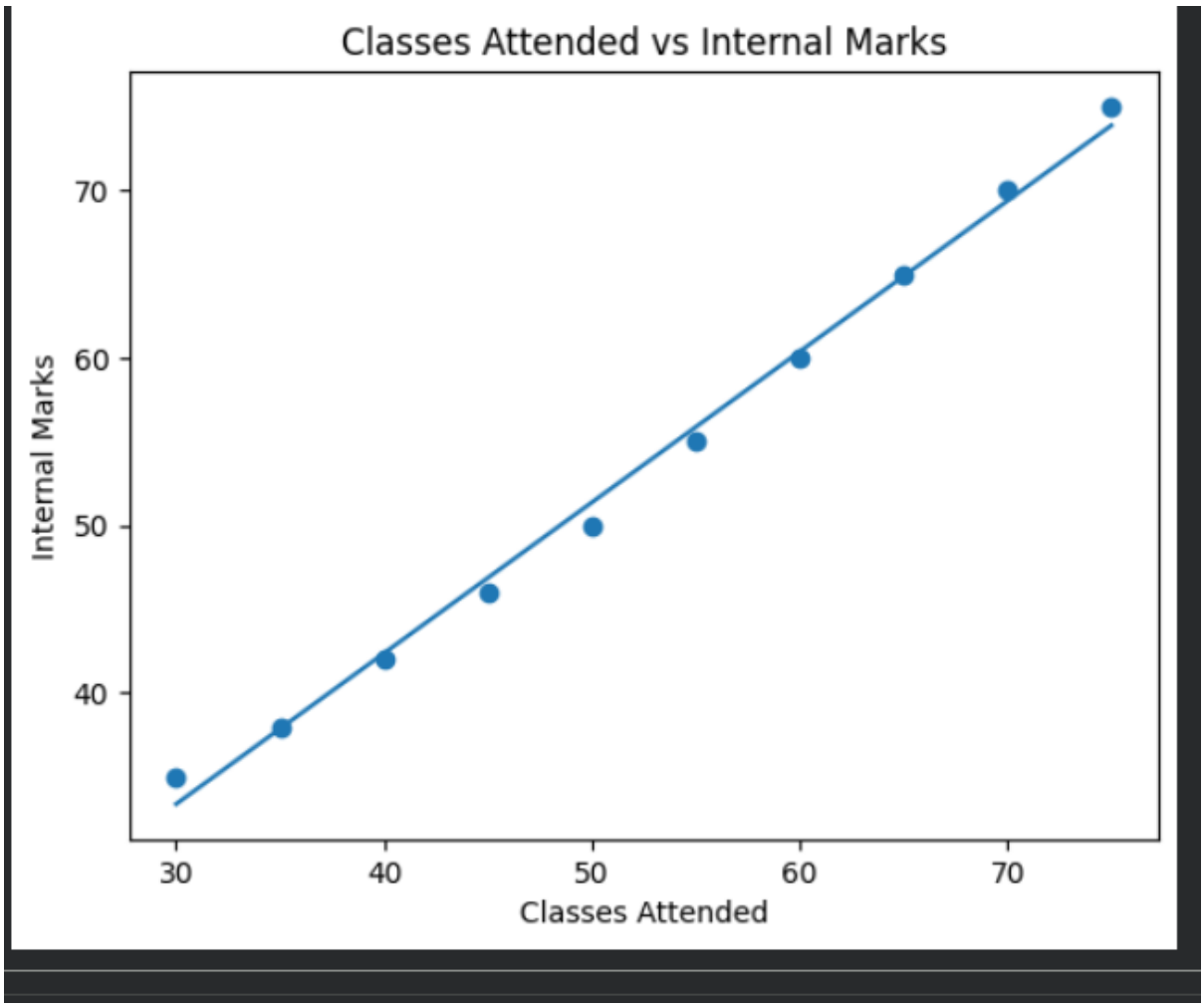
AIM: To implement basic machine learning models using Scikit-learn.

PROGRAM:

```
from ast import Yield
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
data = {
    'Student': ['S1','S2','S3','S4','S5','S6','S7','S8','S9','S10'],
    'Classes_Attended': [30, 35, 40, 45, 50, 55, 60, 65, 70, 75],
    'Internal_Marks': [35, 38, 42, 46, 50, 55, 60, 65, 70, 75]
}
df = pd.DataFrame(data)
print(df)
X = df[['Classes_Attended']]
y = df['Internal_Marks']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=7
)
model=LinearRegression()
model.fit(X_train,y_train)
predictions=model.predict(X_test)
mse = mean_squared_error(y_test, predictions)
print("MSE:", mse)
print("Marks per Class:", model.coef_[0])
print("Base Marks:", model.intercept_)
plt.scatter(X, y)
plt.plot(X, model.predict(X))
plt.xlabel("Classes Attended")
plt.ylabel("Internal Marks")
plt.title("Classes Attended vs Internal Marks")
plt.show()
```

OUTPUT:

```
   Student  Classes_Attended  Internal_Marks
0       S1                30              35
1       S2                35              38
2       S3                40              42
3       S4                45              46
4       S5                50              50
5       S6                55              55
6       S7                60              60
7       S8                65              65
8       S9                70              70
9      S10                75              75
MSE: 0.578125
Marks per Class: 0.9
Base Marks: 6.375
```



Result: Machine learning model implemented successfully using Scikit-learn.

EXP.NO. 2. Data Exploration & Preprocessing

AIM: To explore, clean, and preprocess the dataset using Python techniques.

PROGRAM:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
data = {
    'country': ['india', 'usa', 'india', 'usa', 'uk', 'india'],
    'age': [22, 25, np.nan, 30, 28, 35],
    'salary': [40000, 60000, 50000, np.nan, 72000, 58000],
    'purchased': ['no', 'yes', 'yes', 'no', 'yes', 'yes']}
df = pd.DataFrame(data)
print("---ORIGINAL RAW DATA---")
print(df)
print("\n")
print("---MISSING VALUES COUNT---")
print(df.isnull().sum())
print("\n")
df['age'] = df['age'].fillna(df['age'].mean())
df['salary'] = df['salary'].fillna(df['salary'].mean())
print("---DATA AFTER CLEANING---")
print(df)
print("\n")
df_encoded = pd.get_dummies(df, columns=['country'])
df_encoded['purchased'] = df_encoded['purchased'].map({'yes': 1, 'no': 0})
print("---DATA AFTER ENCODING (All Numbers Now)---")
print(df_encoded)
print("\n")
x = df_encoded.drop('purchased', axis=1)
y = df_encoded['purchased']
scaler = StandardScaler()
x_scaled = scaler.fit_transform(x)
print("---SCALED FEATURES---")
print(x_scaled)
print("\n")
x_train, x_test, y_train, y_test = train_test_split(
    x_scaled, y, test_size=0.2, random_state=42)
print("---TRAINING DATA SHAPE---")
print(x_train.shape)
print("---TEST DATA SHAPE---")
print(x_test.shape)
```

OUTPUT:

```
---ORIGINAL RAW DATA---
  country   age   salary purchased
0   india  22.0  40000.0        no
1     usa  25.0  60000.0       yes
2   india   NaN  50000.0       yes
3     usa  30.0      NaN        no
4      uk  28.0  72000.0       yes
5   india  35.0  58000.0       yes


---MISSING VALUES COUNT---
country      0
age          1
salary       1
purchased    0
dtype: int64


---DATA AFTER CLEANING---
  country   age   salary purchased
0   india  22.0  40000.0        no
1     usa  25.0  60000.0       yes
2   india  28.0  50000.0       yes
3     usa  30.0  56000.0        no
4      uk  28.0  72000.0       yes
5   india  35.0  58000.0       yes


---DATA AFTER ENCODING (All Numbers Now)---
    age   salary  purchased  country_india  country_uk  country_usa
0  22.0  40000.0          0           True       False        False
1  25.0  60000.0          1          False       False         True
2  28.0  50000.0          1           True       False        False
3  30.0  56000.0          0          False       False         True
4  28.0  72000.0          1          False        True        False
5  35.0  58000.0          1           True       False        False


---SCALED FEATURES---
[[-1.48461498 -1.6444529   1.         -0.4472136  -0.70710678]
 [-0.74230749  0.41111323 -1.         -0.4472136   1.41421356]
 [ 0.         -0.61666984  1.         -0.4472136  -0.70710678]
 [ 0.49487166  0.         -1.         -0.4472136   1.41421356]
 [ 0.          1.6444529  -1.          2.23606798 -0.70710678]
 [ 1.73205081  0.20555661  1.         -0.4472136  -0.70710678]]
```

Result: Data cleaned and preprocessed successfully for further analysis.

EXP.NO.3. Linear Regression

AIM: To implement regularized linear regression (Ridge/Lasso) for prediction.

PROGRAM:

```
from sklearn.linear_model import LinearRegression,Ridge,Lasso
from sklearn.metrics import mean_squared_error
from sklearn import datasets
from sklearn.model_selection import train_test_split

diabetes=datasets.load_diabetes()
x_train, x_test, y_train, y_test=train_test_split(diabetes.data,diabetes.target,test_size=0.2,random_state=42)
lr=LinearRegression().fit(x_train,y_train)
ridge=Ridge(alpha=1.0).fit(x_train,y_train)
lasso=Lasso(alpha=0.1).fit(x_train,y_train)
print(f"Linear Regression MSE:{mean_squared_error(y_test,lr.predict(x_test)):.2f}")
print(f"Ridge Regression MSE:{mean_squared_error(y_test,ridge.predict(x_test)):.2f}")
print(f"Lasso Regression MSE:{mean_squared_error(y_test,lasso.predict(x_test)):.2f}")
```

```
Linear Regression MSE:2900.19
Ridge Regression MSE:3077.42
Lasso Regression MSE:2798.19
```

Result: Model trained successfully and predictions were obtained.

EXP.NO.4. Naive Bayes Classifier

AIM: To implement Naive Bayes algorithm for classification tasks.

PROGRAM:

```python
import pandas as pd
from scipy.stats import scoreatpercentile
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
data = {
    'StudyHours': [5,2,8,1,6,3,7,4],
    'Attendence': [80,50,90,40,85,60,88,65],
    'Result': ['pass','fail','pass','fail','pass','fail','pass','fail']
}
df = pd.DataFrame(data)
X = df[['StudyHours', 'Attendence']]
y = df['Result']
X_train, X_test, y_train, y_test = train_test_split(
    *arrays: X, y, test_size=0.25, random_state=42)
model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Predicted Results:", y_pred)
print("Actual Results:", y_test.values)
print("Accuracy:", accuracy )
```

OUTPUT:

```
Predicted Results: ['fail' 'fail']
Actual Results: ['fail' 'fail']
Accuracy: 1.0

Process finished with exit code 0
```

Result: Classification performed successfully with Naive Bayes algorithm.

EXP.NO.5. Regularized Logistic Regression

AIM: To implement logistic regression with regularization for classification.

PROGRAM:

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
data = load_breast_cancer()
X = data.data
y = data.target
X_train, X_test, y_train, y_test = train_test_split( *arrays: X, y, test_size=0.2, random_state=42)
scaler= StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
normal_model = LogisticRegression(C=1e6, penalty='l2', solver='liblinear')
normal_model.fit(X_train_scaled, y_train)
normal_pred = normal_model.predict(X_test_scaled)
normal_acc = accuracy_score(y_test, normal_pred)
regularized_model = LogisticRegression(C=0.1, penalty='l2', solver='liblinear')
regularized_model.fit(X_train_scaled, y_train)
reg_pred = regularized_model.predict(X_test_scaled)
reg_acc = accuracy_score(y_test, reg_pred)
print("Normal Logistic Accuracy:", normal_acc * 100)
print("Regularized Logistic Accuracy:", reg_acc * 100)
plt.figure()
plt.plot( *args: normal_model.coef_.flatten(), label='Normal Logistic')
plt.plot( *args: regularized_model.coef_.flatten(), label='Regularized Logistic')
plt.legend()
plt.title("Coefficient Comparison")
plt.xlabel("Feature Index")
plt.ylabel("Coefficient Value")
plt.show()
plt.figure()
models = ['Normal Logistic', 'Regularized Logistic']
```

OUTPUT:

```
Normal Logistic Accuracy: 93.85964912280701
Regularized Logistic Accuracy: 99.12280701754386
```

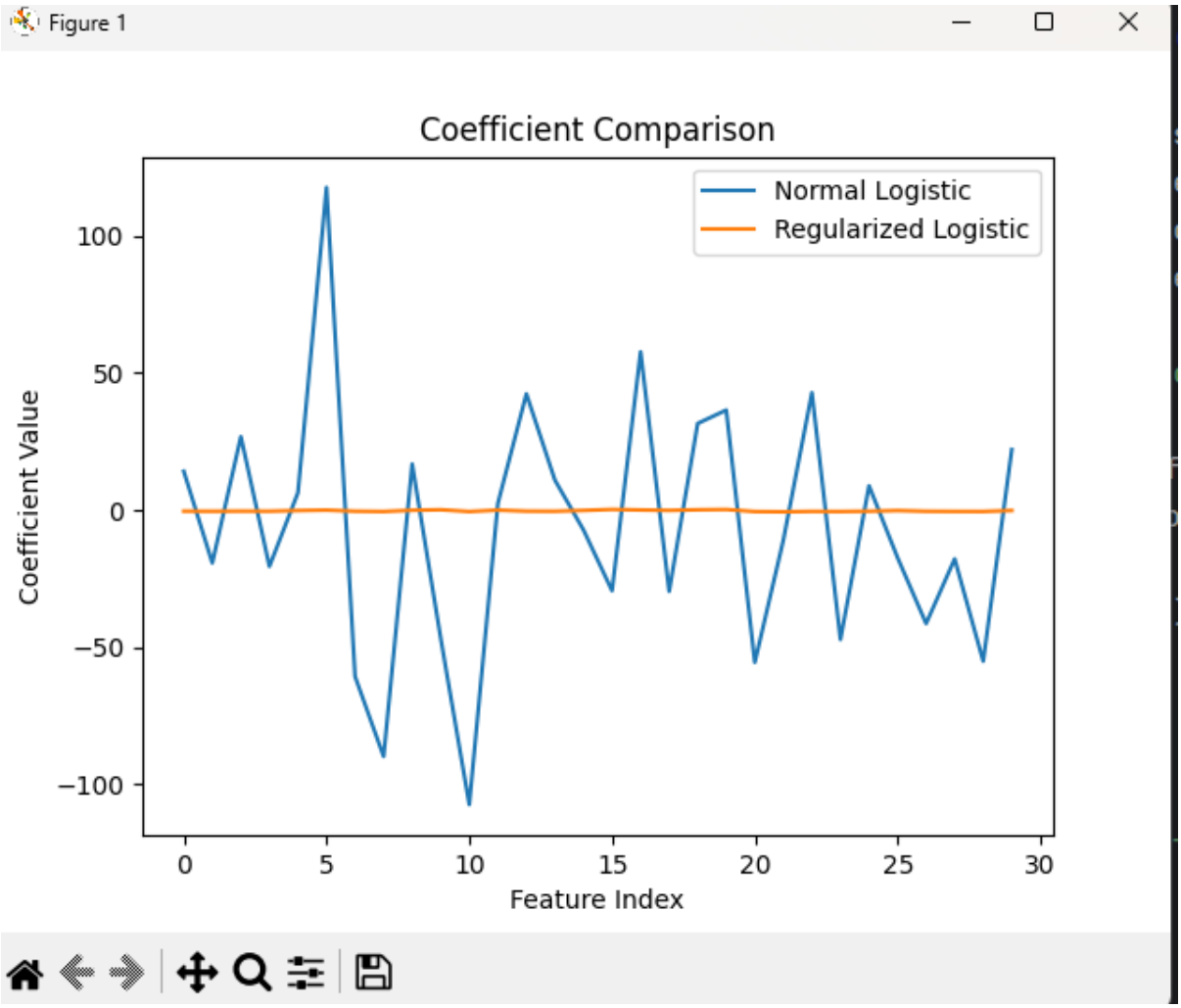


Figure 1
Coefficient Comparison
Normal Logistic
Regularized Logistic
Coefficient Value
100
50
0
−50
−100
0
5
10
15
20
25
30
Feature Index

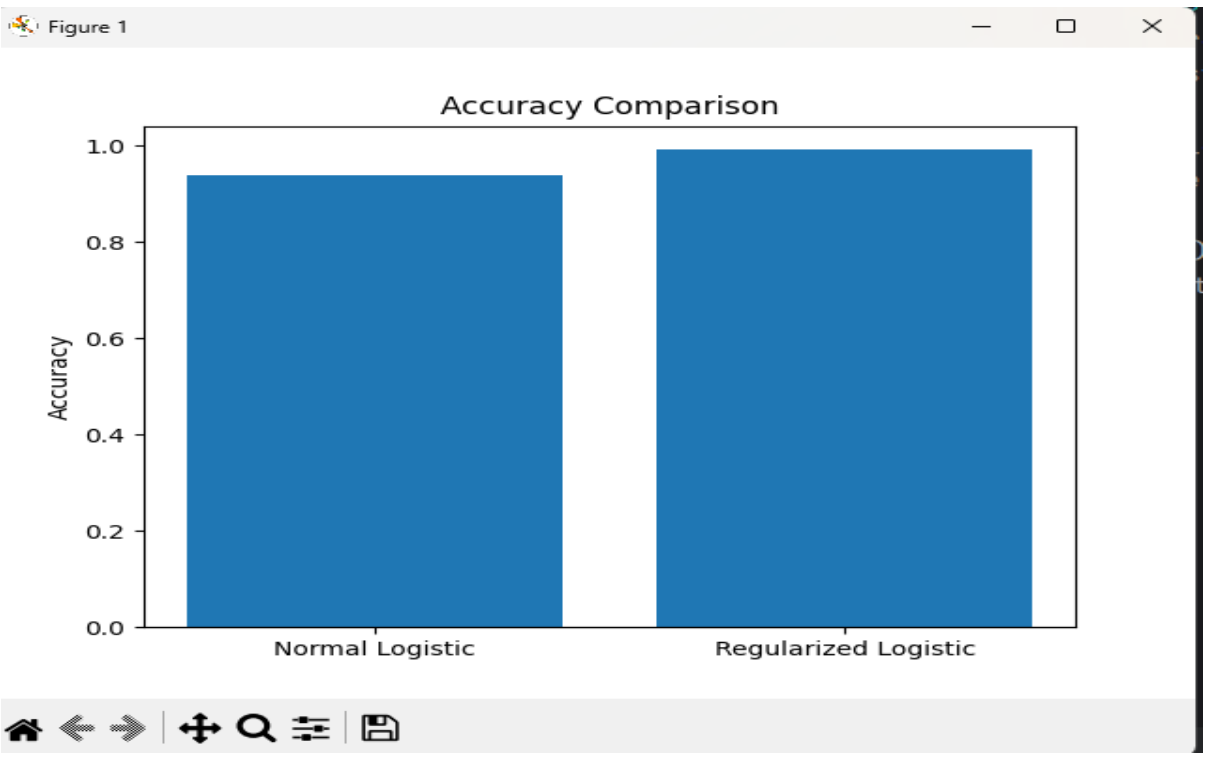


Figure 1
Accuracy Comparison
Accuracy
1.0
0.8
0.6
0.4
0.2
0.0
Normal Logistic
Regularized Logistic

Result: Logistic regression model implemented with regularization successfully.

EXP.NO.6. Ensemble Techniques

AIM: To build models using ensemble methods like bagging and boosting.

PROGRAM:

```
from sklearn import datasets
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, StackingClassifier
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
iris = datasets.load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    *arrays: iris.data, iris.target, test_size=0.2, random_state=42)
rf = RandomForestClassifier(n_estimators=100).fit(X_train, y_train)
ada = AdaBoostClassifier(n_estimators=100).fit(X_train, y_train)
estimators = [
    ('rf', RandomForestClassifier()),
    ('svc', SVC(probability=True))]
stack = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression()
).fit(X_train, y_train)
print(f"Random Forest Accuracy: {accuracy_score(y_test, rf.predict(X_test)):.2f}")
print(f"AdaBoost Accuracy: {accuracy_score(y_test, ada.predict(X_test)):.2f}")
print(f"Stacking Accuracy: {accuracy_score(y_test, stack.predict(X_test)):.2f}")
```

OUTPUT:

```
Random Forest Accuracy: 1.00
AdaBoost Accuracy: 0.93
Stacking Accuracy: 1.00

Process finished with exit code 0
```

Result: Ensemble models built and improved prediction accuracy.

EXP.NO.7. Decision Tree Methods

AIM: To build and evaluate models using Decision Tree algorithms.

PROGRAM:

```
from sklearn import datasets
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
iris = datasets.load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    *arrays: iris.data, iris.target, test_size=0.2, random_state=42)
dt_clf = DecisionTreeClassifier(max_depth=3).fit(X_train, y_train)
y_pred = dt_clf.predict(X_test)
print(f"Decision Tree Classifier Accuracy: {accuracy_score(y_test, y_pred):.2f}")
print("Feature Importances:", dt_clf.feature_importances_)
```

OUTPUT:

```
Decision Tree Classifier Accuracy: 1.00
Feature Importances: [0.         0.         0.39426873 0.60573127]

Process finished with exit code 0
```

Result: Decision tree model constructed and used for classification successfully.

EXP.NO.8. Support Vector Machine (SVM)

AIM: To implement SVM with different kernel functions for classification.

PROGRAM:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris
data = load_iris()
X = data.data
y = data.target
X_train, X_test, y_train, y_test = train_test_split( *arrays: X, y, test_size=0.2)
model_linear = SVC(kernel='linear')
model_linear.fit(X_train, y_train)
pred_linear = model_linear.predict(X_test)
acc_linear = accuracy_score(y_test, pred_linear)
print("Linear Kernel Accuracy:", acc_linear)
model_poly = SVC(kernel='poly', degree=3)
model_poly.fit(X_train, y_train)
pred_poly = model_poly.predict(X_test)
acc_poly = accuracy_score(y_test, pred_poly)
print("Polynomial Kernel Accuracy:", acc_poly)
model_rbf = SVC(kernel='rbf')
model_rbf.fit(X_train, y_train)
pred_rbf = model_rbf.predict(X_test)
acc_rbf = accuracy_score(y_test, pred_rbf)
print("RBF Kernel Accuracy:", acc_rbf)
```

OUTPUT:

```
Linear Kernel Accuracy: 0.9333333333333333
Polynomial Kernel Accuracy: 0.9333333333333333
RBF Kernel Accuracy: 0.8666666666666667

Process finished with exit code 0
```

Result: SVM model implemented with different kernels and results obtained.

EXP.NO.9. K-Nearest Neighbors (KNN)

AIM: To implement KNN algorithm for classification and prediction.

PROGRAM:

```python
import pandas as pd
import os
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
print("Current Directory:", os.getcwd())
data = pd.read_csv(r"D:\DSTLAB_11249A354\mobile_price.csv")
print(data.head())
X = data.drop( labels: "price_range", axis=1)
y = data["price_range"]
X_train, X_test, y_train, y_test = train_test_split(
    *arrays: X, y, test_size=0.2, random_state=42
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
print("\nAccuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

OUTPUT:

```
   battery_power  blue  clock_speed  ...  touch_screen  wifi  price_range
0            842     0          2.2  ...             0     1            1
1           1021     1          0.5  ...             1     0            2
2            563     1          0.5  ...             1     0            2
3            615     1          2.5  ...             0     0            2
4           1821     1          1.2  ...             1     0            1

[5 rows x 21 columns]

Accuracy: 0.53

Confusion Matrix:
 [[76 24  5  0]
 [25 41 22  3]
 [ 6 41 34 11]
 [ 2 13 36 61]]

Classification Report:
              precision    recall  f1-score   support

           0       0.70      0.72      0.71       105
           1       0.34      0.45      0.39        91
           2       0.35      0.37      0.36        92
           3       0.81      0.54      0.65       112

    accuracy                           0.53       400
   macro avg       0.55      0.52      0.53       400
weighted avg       0.57      0.53      0.54       400


Process finished with exit code 0
```

Result: KNN algorithm implemented and classification results obtained.

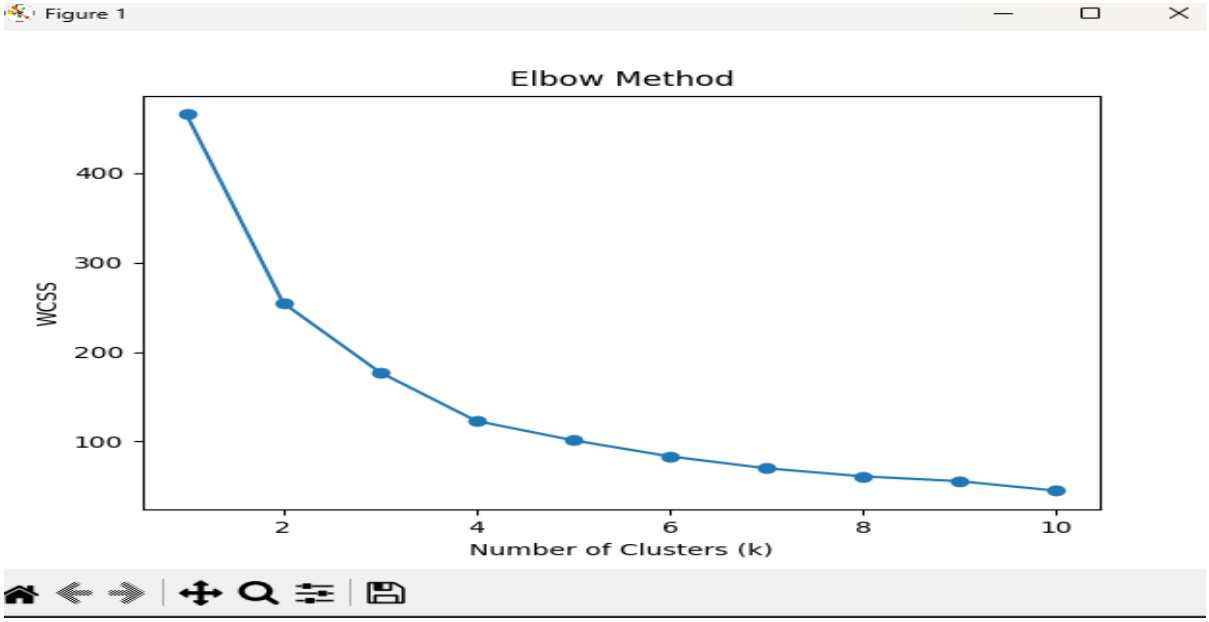
EXP.NO.10. K-Means with PCA & Elbow Method

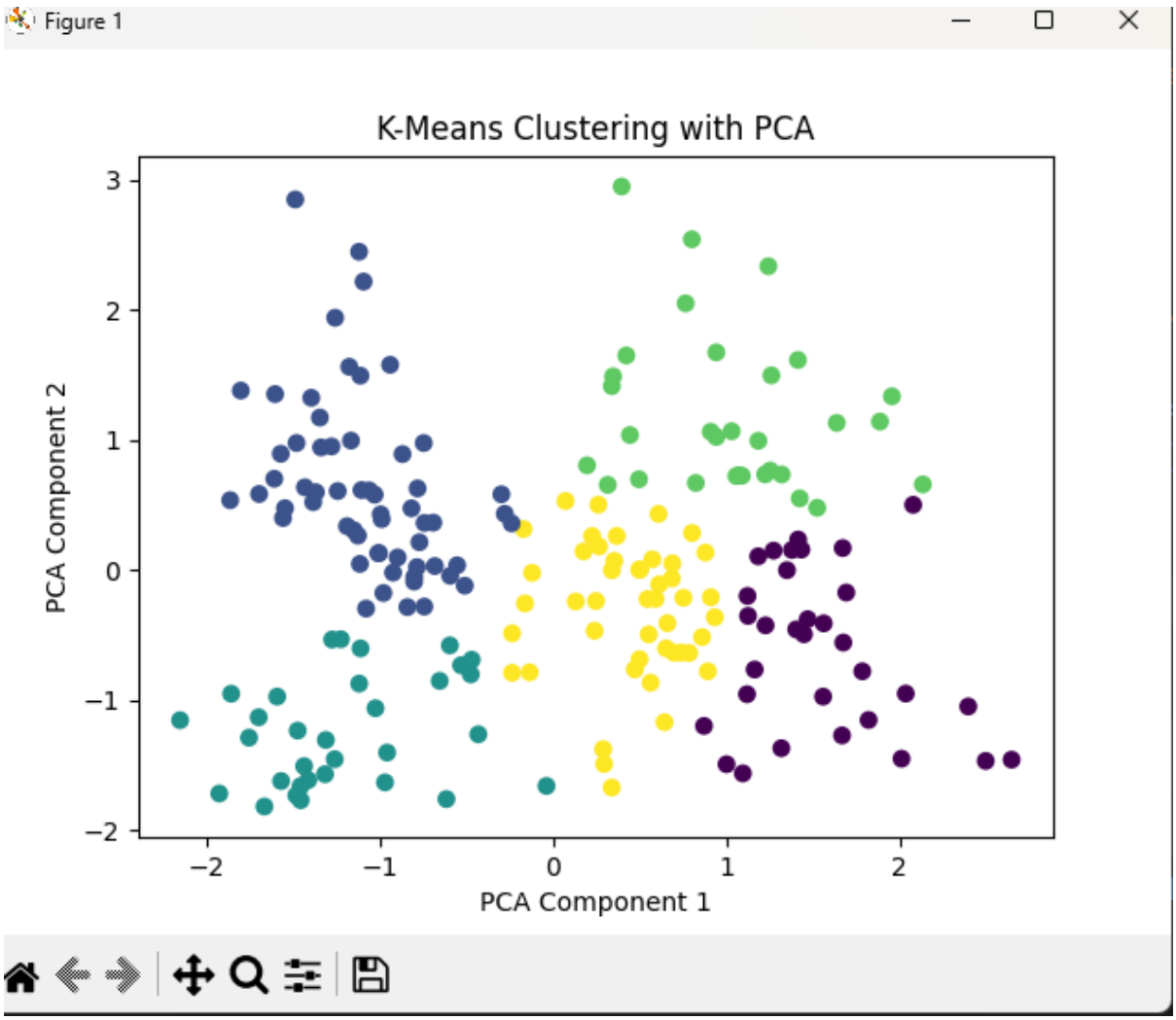
AIM: To perform clustering using K-Means and optimize clusters using PCA and Elbow method.

PROGRAM:

```python
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
data = pd.read_csv("Mall_Customers.csv")
X = data[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
print("Explained Variance Ratio:", pca.explained_variance_ratio_)
wcss = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, random_state=0)
    kmeans.fit(X_pca)
    wcss.append(kmeans.inertia_)
plt.plot( *args: range(1, 11), wcss, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters (k)")
plt.ylabel("WCSS")
plt.show()
kmeans = KMeans(n_clusters=5, random_state=0)  # Fixed typo: N_cluster -> n_clusters
clusters = kmeans.fit_predict(X_pca)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clusters, cmap='viridis')
plt.title("K-Means Clustering with PCA")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.show()
```

OUTPUT:





Result: Optimal clusters identified and data clustered successfully.